Floating-Point Rounding and Arithmetic Operations

Rounding Methods

In floating-point arithmetic, exact representation of all real numbers is impossible due to finite bit precision. When a computation result cannot be exactly represented in the target format, rounding becomes necessary to **approximate the true mathematical result**.

Challenges:

- Infinite real numbers must map to finite floating-point representations.
- Rounding introduces controlled approximation errors.
- Different rounding strategies optimize for different mathematical properties.

Truncation (Round toward Zero)

- **Direction**: Always toward zero on the number line.
- **Behavior**: Remove the fractional portion without regard to magnitude.
- **Characteristics**:
  - Positive numbers round down (toward smaller magnitude).
  - Negative numbers round up (toward smaller magnitude).
  - Reduces absolute value, never increases it.
  - Systematic bias toward zero.

Round Up (Round toward +∞)

- **Direction**: Always toward positive infinity.
- **Behavior**: Select the smallest representable value ≥ the true result.
- **Characteristics**:
  - All numbers round toward positive infinity.
  - Never decreases value, may increase it.
  - Systematic upward bias.
  - Exact values remain unchanged.

Round Down (Round toward -∞)

- **Direction**: Always toward negative infinity.

- **Behavior**: Select the largest representable value ≥ the true result.
- **Characteristics**:
  - All numbers round toward negative infinity.
  - Never increases value, may decrease it.
  - Systematic downward bias.
  - Exact values remain unchanged.

Round to Nearest, Ties to Even (Default Rounding Mode)

Basic Steps:

1. **Round**: Find closest representable values above and below the true result.
   - If one is clearly closer: Select that value.
   - If exactly equidistant (tie): Select the value with an even least significant digit. If the two closest even representations are equidistant, use the one leaning towards +∞.

Sample Rounding Problems

$+67.490550_{10}$	7 decimal digits	6 decimal digits	5 decimal digits	4 decimal digits
Truncate	+67.49055	+67.4905	+67.490	+67.49
Floor	+67.49055	+67.4905	+67.490	+67.49
Ceiling	+67.49055	+67.4906	+67.491	+67.50
RTN-TE	+67.49055	+67.4906	+67.491	+67.49

-101.1101010_{2}	9 decimal digits	8 decimal digits	7 decimal digits	6 decimal digits
Truncate	-101.110101	-101.11010	-101.1101	-101.110
Floor	-101.110101	-101.11011	-101.1110	-101.111
Ceiling	-101.110101	-101.11010	-101.1101	-101.110
RTN-TE	-101.110101	-101.11010	-101.1101	-101.111

FP Arithmetic through Rounding Method

Basic Steps:

- Align exponents to the larger exponent.
- Round the operands using RTN-TE to target precision.
- Perform the arithmetic operation. Normalize the result if necessary

> Example 1: Perform the computation $1.01110111 * 2^5 + 1.1010100 * 2^2$ to 6 bits (use RTN-TE if needed). All answers should be normalized.
>
> Align smaller exponent to larger exponent:
> $1.1010100 * 2^2$ is converted to $0.0011010100 * 2^5$
>
> | | | $Base^{Esp}$ |
> |---|---|---|
> | Operand 1 | 1.01111 | 2^5 |
> | Operand 2 | 0.00111 | 2^5 |
> | Final Sum | 1.10110 | 2^5 |

FP Arithmetic through Guard/Round/Sticky Method

Basic Steps:

- Align exponents to the larger exponent.
- Get the GRS bits to have increased precision.
  - G and R bits are the two next bits. S bit is 1 if there exists a 1 in the rest of the operand.
- Perform arithmetic with extended precision.
- Round final result to target precision.

> Example 1: Perform the computation $1.01110111 * 2^5 + 1.1010100 * 2^2$ to 6 bits (use RTN-TE if needed). All answers should be normalized.
>
> Align smaller exponent to larger exponent:
> $1.1010100 * 2^2$ is converted to $0.0011010100 * 2^5$
>
> | | | G | R | S | $Base^{Esp}$ |
> |---|---|---|---|---|---|
> | Operand 1 | 1.01110 | 1 | 1 | 1 | 2^5 |
> | Operand 2 | 0.00110 | 1 | 0 | 1 | 2^5 |
> | Final Sum | 1.10101 | - | - | - | 2^5 |